Technical Assessment: Multi-Site Website Manager

Introduction

You have **7 calendar days** to build a small product: a website manager and renderer that lets one team run many websites from a shared library of components. We expect about **5–7 hours** of work; please don't spend much more than that. A smaller, well-reasoned submission beats a bigger one.

Timeline: you receive this on Monday 9 November, and the deadline is **9:00 am AEDT, Monday 16 November**. The week includes a weekend so you can fit the work around your commitments. If something comes up, such as illness or a family commitment, tell us and we'll extend the deadline.

The exercise is close to a real problem the Rugby Australia digital team works on. It is not something we will ship or reuse: it exists only so we can see how you think, design and build. We care more about clear decisions and a working core than about feature count.

After you submit, we'll book a 45-minute follow-up session to walk through your solution together (see *How we evaluate*).

The problem

A national sports body runs dozens of websites — national teams, state unions, tournaments, foundations — with one small team. Every site has its own brand, navigation and pages, but most pages are built from the same kinds of blocks: hero banners, news cards, fixture lists, call-to-action panels, profile grids.

Today those pages are stored as **JSON documents** that describe which components appear on a page and with what content. Each website reads its pages and renders them with a shared component library. The JSON is edited through a heavyweight, general-purpose headless CMS that is hard to run, slow to change, and couples the page format to the CMS itself.

We want to explore replacing that with a purpose-built, much simpler **website manager**: a tool for editors to compose pages from reusable components, while the websites keep rendering pages from portable JSON.

What to build

Build a **Next.js** application (one app or a monorepo, your choice) with two parts:

1. **The manager** — where an editor creates pages for a site, adds components to a page, and edits each component's content.
2. **The renderer** — the public websites. Given a site and a URL path, it loads the page JSON and renders it with the shared component library and that site's theme.

Sites themselves come from seed data: you don't need screens to create or edit sites.

You design the data model. These are the concepts we expect it to cover; name and shape them however you like:

Concept	What it is
Site	A website: name, slug or domain, theme. Seeded, not edited in the manager
Page	A route on a site (e.g. `/`, `/about`) with a title and a tree of component instances
Component definition	A reusable component in the library: its type, the props it accepts, and how those props are described so the manager can edit them
Component instance	A component placed on a page, with its prop values
Theme	The per-site design tokens that make the same component look different on different sites

The domain is a sports organisation's websites. Use mock content; no real APIs are needed.

Requirements (must have)

A submission that does these well beats one that does everything partially.

1. **Pages are portable JSON.** A page is stored as a JSON document — a tree of component instances (type, props, children) — that can be saved and rendered without the manager running.
2. **Two required components** (described below), built on a few small building blocks of your own such as Button, Heading and Image.
3. **Components describe their own editable props.** The manager generates the editing form for a component from that description; adding a new component should not require hand-writing a new form.
4. **Manager flows:** create a page on a site, add and remove components on it (including inside a Section), and edit their props. Viewing the rendered public page is enough as a preview.
5. **Multi-site reuse: two seeded sites** use the same components, each with its own theme, and visibly look different.
6. **Rendering:** public pages are server-rendered and resolved by site + path. Unknown paths return a 404.
7. **Persistence:** data is stored in MongoDB, run locally (e.g. Docker) or on a free MongoDB Atlas cluster. The official driver or Mongoose — your choice.
8. **Runs locally** with one documented command (or a short list of them), including a seed script for the two sites.

Required components

Each one tests something different. The props are suggestions: adapt names and shapes, but keep what each component is meant to exercise.

1. Section (container). Groups other components and controls layout. Tests nesting: the page is a tree in the JSON, the renderer and the manager.

- `layout`: `full` | `two-column`
- `background`: `none` | `light` | `brand` (resolved from the site theme)
- children: other components, including another Section

2. Card Grid (rich props). A heading and a list of cards, e.g. “Meet the team”. Tests generating forms for lists, nested objects and optional fields, plus theming.

- `title`: string
- `cards`: list of `{ image: { url, alt }, heading, link?: { label, href } }`

Stretch goals (optional)

Only once the must-haves work. Doing none of these is completely fine; we'll talk through how you'd approach them in the follow-up.

- **Deploy to Vercel** with the manager and both sites at public URLs.
- **A data-driven component**, e.g. “Latest news”, whose editor sets options (source, limit) and whose data comes from a mock API at render time.
- **Reordering components** on a page, or a live preview inside the manager.
- **Draft and publish**, so the public site only shows published versions.
- **Tests** that cover the parts you consider risky.

Constraints and open decisions

A few things are fixed; everything else is your call, and explaining your choices is part of the exercise.

Fixed	Your decision
Next.js and TypeScript	App Router or Pages Router
Pages stored as portable JSON	The shape of the JSON and the data model
Components describe their editable props	How they describe them (Zod, JSON Schema, a custom format…)
MongoDB for storage	Driver or ODM, document modelling
No off-the-shelf headless CMS or page builder (Keystone, Strapi, Sanity, Payload, Builder.io, Puck, etc.)	UI and utility libraries (component kits, form libraries) are fine
	Styling approach, project structure, and component architecture (atomic design or another approach, with your reasoning)

Deliverables and submission

Send us a link to andres.solarte@rugby.com.au to a Git repository (GitHub, GitLab or Bitbucket; private with access granted to us is fine) by the deadline. Commit as you go: we read the history.

The repository should contain:

- ☐ The source code and a seed script for the **two sites**, with a page or two each.
- ☐ A short **README** covering:
  - ☐ How to run it.
  - ☐ Your page JSON format, with one example document.
  - ☐ How to add a new component.

- ☐ Key decisions and trade-offs, roughly how many hours you spent, and which AI tools you used, if any.

How we evaluate

We look at the whole submission against these areas, weighted roughly as shown:

Area	Weight	What we look for
Architecture	30%	Clear separation between page data, component definitions, the manager and the renderer; a data model that would hold up with 30+ sites
Component and schema design	30%	How components declare their props, how themes vary them, and how easy it is to add a new one
Product and editor experience	15%	Can a non-developer create a page and fill it with components without help?
Code quality	10%	Readable, typed, consistent code; sensible commit history
Communication	15%	README quality, honest trade-offs, and how you explain your work in the follow-up

Follow-up session (45 minutes). You'll run the project on your machine and share your screen. About 15 minutes for you to walk us through the product and your main decisions, about 20 minutes pairing on a small change (for example, adding a new prop to a component and editing it in the manager) and discussing how you'd approach a stretch goal, and 10 minutes for your questions. There is nothing extra to prepare.

AI tools. You may use AI assistants. You are responsible for every line you submit, and in the follow-up we will ask you to explain and change your code without them.

FAQ

Do the websites need to look polished? No. Two clearly different themes matter more than pixel-perfect design.

Do I need authentication? No. A single unauthenticated editor is fine.

Can I use a component library like shadcn/ui or Radix? Yes, for the manager and as building blocks. Your site components and their prop schemas should be your own.

What if I run out of time? Stop, and write in the README what's missing and how you'd build it. That counts.

Will this cost me anything? No. MongoDB runs locally in Docker, or on Atlas's free M0 cluster. Keep secrets out of the repo: use environment variables and list them in a `.env.example`.

Something is unclear. Make a reasonable assumption, note it in the README, and carry on. You can also email [contact name / email] — we aim to reply within one business day.